CS319

Fourth Deliverable

Team 6

Section 1

Bora Yetkin - 22203661

Erdem Pülat - 22103566

Halil Avcı - 22003476

Ramiz Arda Ünal - 22202554

Tuna Göksal - 22203827

Class Diagram With Design Patterns

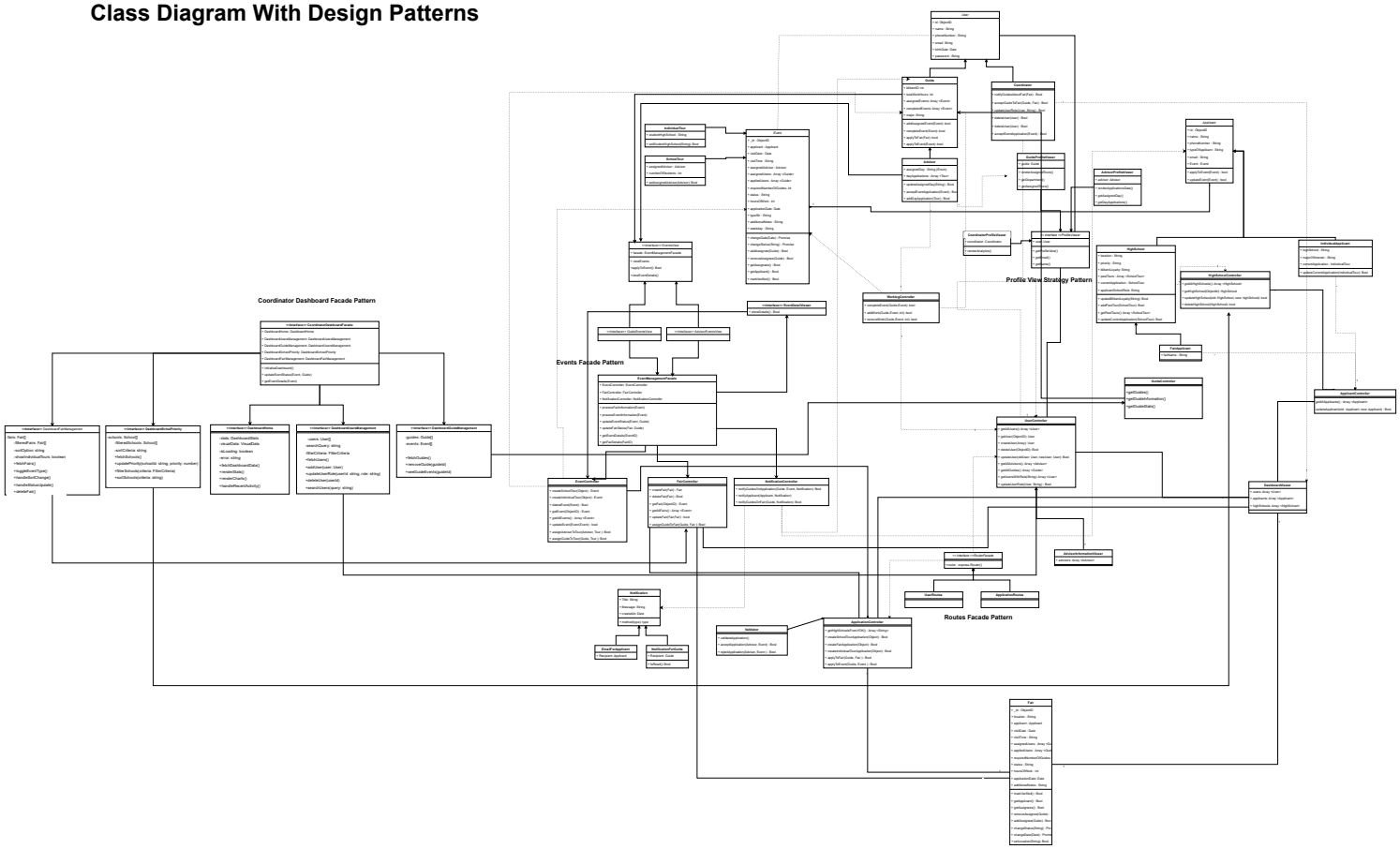
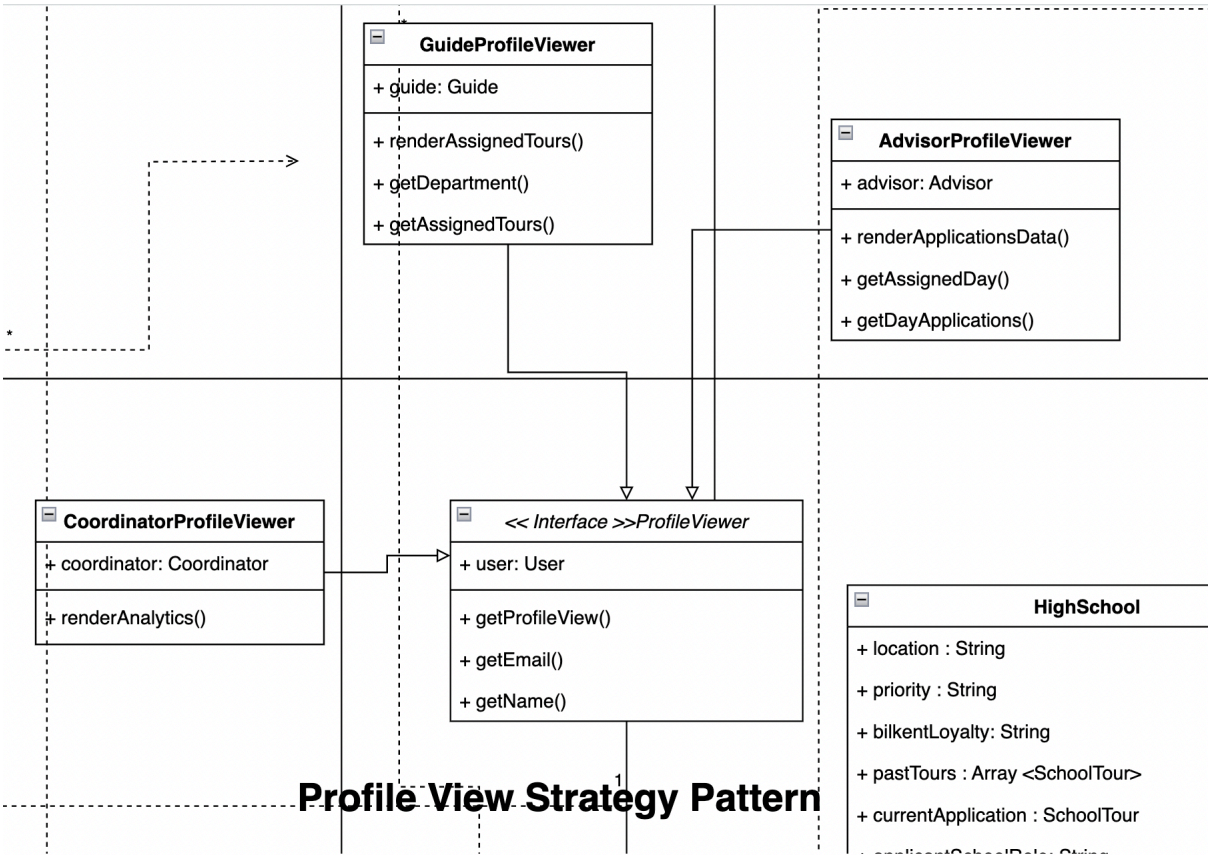


Diagram drive link:
https://drive.google.com/file/d/1B2gCDLTFPIqVUFO-7LoxR3yizfcEN9fB/view

Design Patterns

1 - Strategy Pattern:

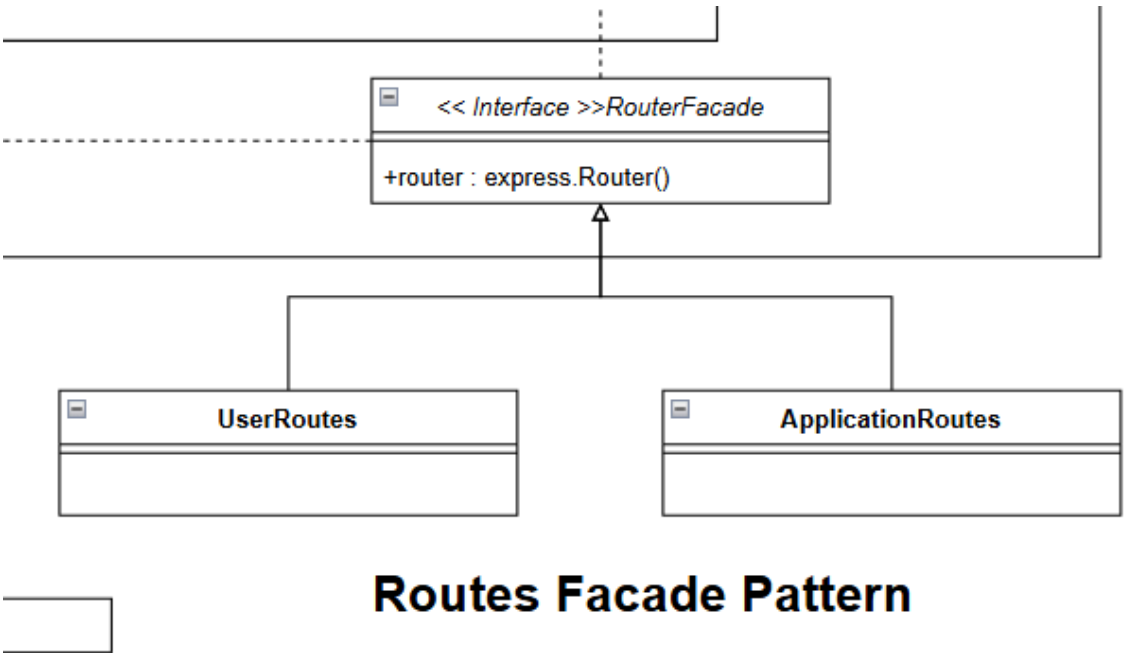
1.1 Profile View Strategy Pattern



The Strategy Pattern allows the profile view to dynamically adapt based on the user type, offering a tailored experience for different roles. This design ensures shared functionalities, like updating an email address, remain consistent across all users, while role-specific features can be independently customized. The system avoids cluttering a single controller with role-specific logic by encapsulating behaviors such as resetting passwords or changing roles into interchangeable strategies. This modular approach enhances maintainability and simplifies testing, as issues in one user type's behavior do not affect others. Additionally, it supports future scalability by enabling new user roles to be seamlessly integrated without altering existing strategies.

2 - Facade Pattern:

2.1 Router Facade

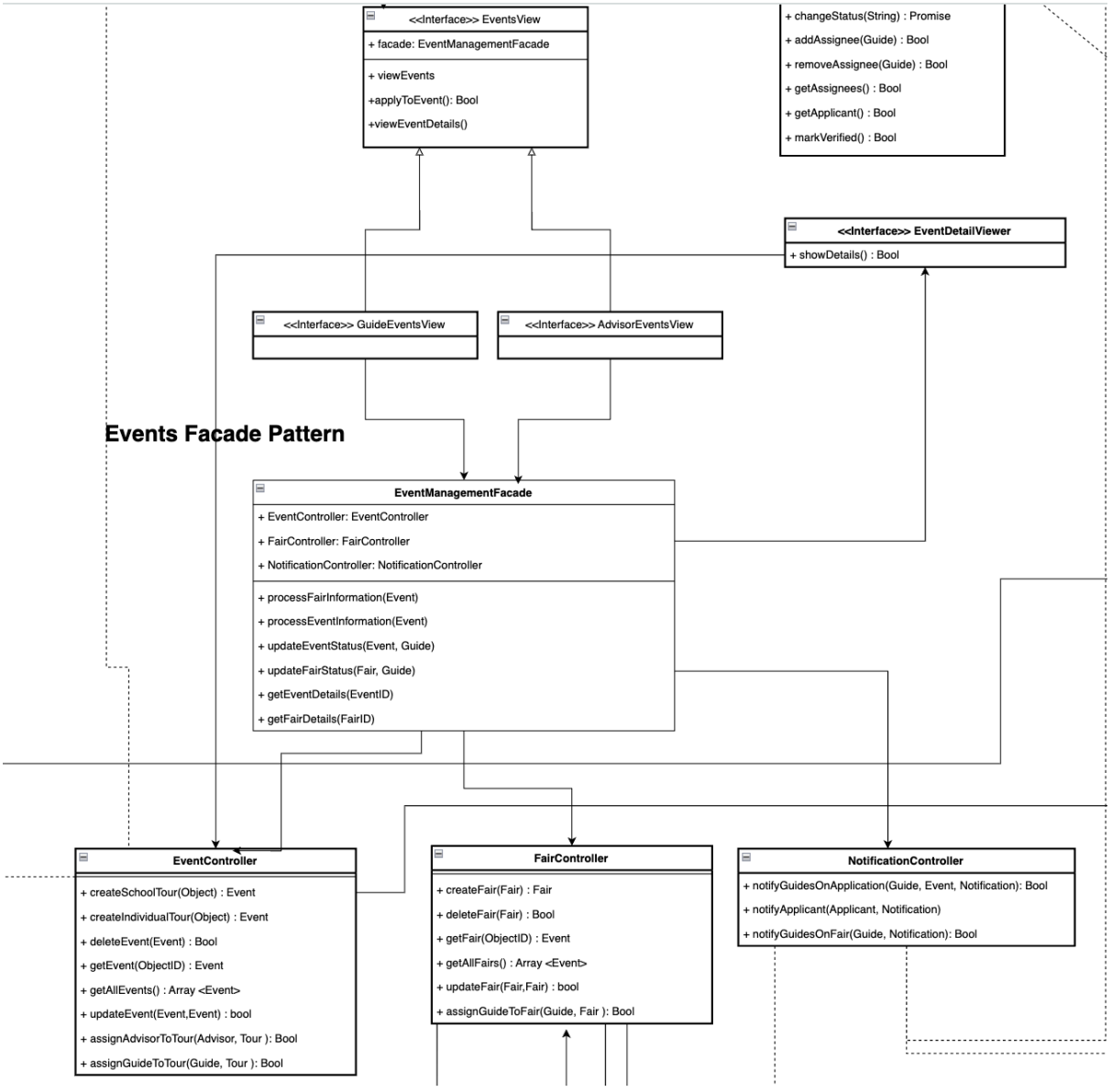


Routes Facade Pattern

The **RouterFacade** serves as the facade in this structure. It is of significant importance to manage the routing logic throughout the project by creating a centralized access point to simplify interactions with the application's routing system. The facade achieves this by delegating tasks to the relevant subsystems, namely UserRoutes and ApplicationRoutes.

Instead of requiring clients to deal with the problems of managing many routes, the facade proposes a unified router: express.Router() method, which encapsulates all the necessary routing logic. This allows the client to access the functionality of both UserRoutes and ApplicationRoutes and other routes that exist for each of the controllers, which are not present here for the sake of simplicity. For example, it handles user-specific routing through UserRoutes while managing application-wide routes through ApplicationRoutes. This approach keeps the routing structure clean, modular, and easy to maintain and ensures that clients interact only with the facade rather than the underlying route configurations. This also helps us, the developers, to use the express ecosystem without understanding every part of the underlying subsystems of the library.

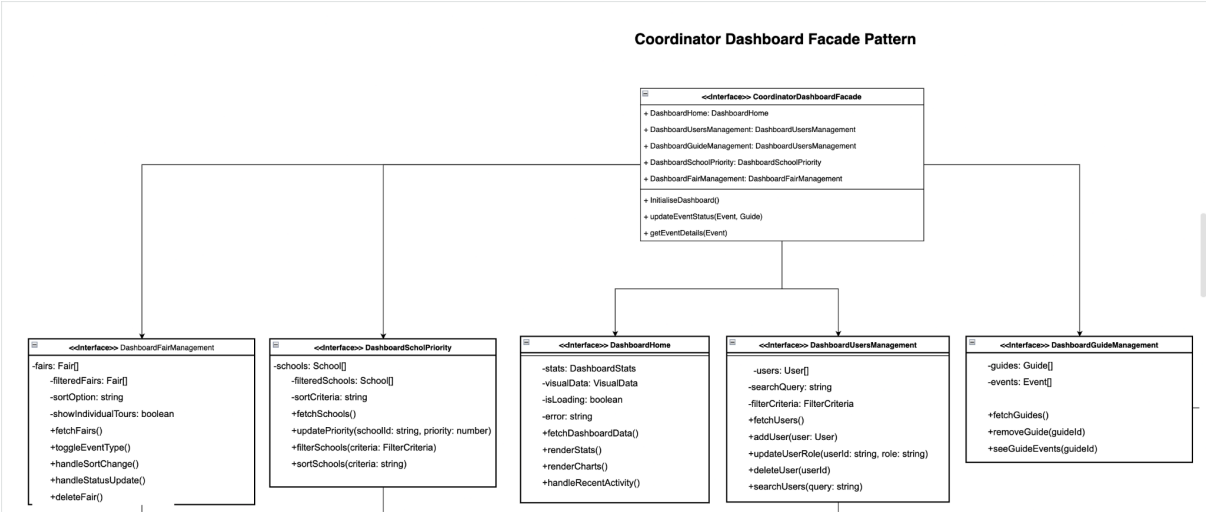
2.2 Event Management Facade



EventManagementController serves as the facade of this structure. It holds great importance for the advisor and coordinator user types as these classes provide the main structure for them to manipulate tours and fairs. It creates a central access point to simplify the interactions with the system. The facade anchovies this by delegating tasks to the relevant subsystems. These subsystems are the fair

controller, event controller, and notification controller. So facade allows them to client use facade's methods such as **processEventManagement(Event)**, which handles event processing; **updateEventStatus(Event, Guide)**, which updates the status of an event; and **getEventDetails(Event)**, which records event-related information rather than requiring the client to interact with each of these controllers individually. The fair version of these methods also exists with similar functionality.

2.3 Coordinator Dashboard Facade



The Coordinator Dashboard is the most important part of our system for the coordinator users. As such, we needed to provide as much functionality as possible while having a compartmentalized and sustainable codebase for future additions. The **Facade** design pattern is a perfect fit for this aim, with a **CoordinatorDashboardFacade** interface that oversees the coordinator dashboard library. The client can access the functionality with the help of the interface only, without needing to call or know about the complex operations in the background. As such, adding new functionality to the library does not affect the other parts of the library. While the **Router** facade allowed us to interact with the **express** ecosystem without knowing the insides, **CoordinatorDashboardFacade** simplifies client interactions with a subsystem we wrote. The insides of the subsystem are divided for each use case: fair management, school management, user management, and guide management. Lastly, we have planned **Dashboard Home** as a nice page that

visualizes data and gives general information, and for now does not interact with other parts. In the future, associations may be added. Further additions or editions can be made with ease.